



Actividad práctica en parejas

Objetivo: tomar una batalla en consola que tiene tres problemas de estructura, una escuadra que no se puede tratar como un soldado, el equipamiento escrito como una subclase por combinación y cada ataque resuelto paso a paso contra el subsistema de combate, y resolverlos con Composite, Decorator y Facade. Después, darle un estandarte a una escuadra entera sin tocar ninguna clase existente.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es *14-estructurales/actividad*.

Todo está en el paquete *modelo*. *Combatiente* es la interfaz de lo que pelea, con *descripcion()*, *ataque()* y *defensa()*, y *Soldado* la implementa. Ninguna de las dos se modifica. *Escuadra* es un grupo de soldados, pero no implementa *Combatiente*. El equipamiento son tres subclases de *Soldado*, una por combinación. *Dado*, *ReglasCombate* y *Bitacora* forman el subsistema de combate y tampoco se modifican. *Programa* arma los dos bandos, resuelve tres ataques a mano e imprime la bitácora.

Parte 1: ver el problema

1. Ejecutar *Programa* y leer la bitácora. Después leer *ataqueTotal()* al final de *Programa*. ¿Por qué la lista del ejército es de *Object*? ¿Qué habría que agregar a esa función si el ejército tuviera una escuadra adentro de otra?
2. Intentar agregar una escuadra a otra, *rohan.agregar(new Escuadra("Jinetes"))*. No compila. Borrar la línea.
3. Contar cuántas clases harían falta para agregar una armadura que sume defensa, con todas las combinaciones con espada y escudo. Pensar también cómo se le daría una espada a toda la escuadra de Rohan.
4. Mirar los tres bloques de ataque en *Programa*. ¿Cuántas líneas cambian de un bloque a otro? ¿Cuántas clases del subsistema conoce *Programa*?

Parte 2: los tres patrones

Un TODO por vez, en orden. Cada TODO indica en qué archivos se trabaja.



- TODO 1 (*Escuadra.java*, *Programa.java*): *Escuadra* implementa *Combatiente* y guarda una lista de *Combatiente* en vez de una de *Soldado*. *agregar()* recibe un *Combatiente* y devuelve *false* si es *null* o si es la propia escuadra. *ataque()* y *defensa()* suman los de sus miembros, y *descripcion()* devuelve el nombre seguido de las descripciones de los miembros. En *Programa*, el ejército pasa a ser *new Escuadra("Ejercito")*, la función *ataqueTotal()* se borra, y donde se usaba se llama a *ejercito.ataque()*.
- TODO 2 (clases nuevas, *Programa.java*): una clase abstracta *Equipamiento* que implementa *Combatiente*, recibe por constructor el *Combatiente* que lo porta y le delega los tres métodos. *Espada* suma 5 al ataque y *Escudo* suma 4 a la defensa, cada una agregando " con espada" o " con escudo" a la descripción. Las tres clases *SoldadoCon...* se borran. En *Programa*, Aragorn se arma envolviendo un *Soldado* en una *Espada* y eso en un *Escudo*, y la variable pasa a ser de tipo *Combatiente*.
- TODO 3 (clase nueva, *Programa.java*): una clase *Partida* que recibe por constructor el *Dado*, las *ReglasCombate* y la *Bitacora*, y tiene un único método *int atacar(Combatiente atacante, Combatiente defensor)* que hace los cuatro pasos de un ataque y devuelve el daño. En *Programa*, los tres bloques de ataque pasan a ser tres llamadas a *atacar()*.

Con los tres TODO hechos, la bitácora de los dos primeros ataques es la misma que al principio, y el tercero muestra la descripción completa del ejército.

Parte 3: el estandarte

- TODO 4: agregar *Estandarte*, un equipamiento que suma 3 al ataque. En *Programa*, en vez de agregar *rohan* al ejército, agregar *new Estandarte(rohan)*. El ataque total del ejército tiene que subir 3.

Antes de seguir, comprobar que ni *Escuadra* ni *Equipamiento* cambiaron para que entrara el estandarte, y que *Programa* ya no importa *Dado* ni *ReglasCombate* salvo para crearlos.

Para discutir en la pareja y anotar:

- El estandarte envuelve a una escuadra, que contiene soldados equipados. ¿Qué patrón hace posible cada nivel de esa mezcla? ¿Por qué funciona sin que ninguno sepa del otro?
- *Escuadra.agregar()* rechaza a la propia escuadra. ¿Alcanza para evitar ciclos? ¿Qué pasa si A contiene a B y se intenta agregar A a B?
- Si el resto del juego guardó una referencia al *Soldado* de Aragorn antes de darle la espada, ¿esa referencia ve la espada? ¿Qué consecuencia tiene para el proyecto?
- *Partida* recibe el dado, las reglas y la bitácora por constructor. ¿Qué permite eso a la hora de probar *atacar()*, por ejemplo con un dado que siempre saca 6?
- En el proyecto integrador, ¿qué se agrupa y se trata como una unidad? ¿Qué ganan y pierden los personajes durante la partida? ¿Cuántas clases del modelo llama hoy el controlador para completar una acción?